How we built the ÖSO pomade site

Barbelo Agency · reverse engineering, step by step

Live site: https://oso-pomade.pages.dev

The idea: a product site where **the jar never leaves the screen**. Everything moves around it: the room, the light, the hands, the people.

The recipe in one look

Piece	What it is	Tool
Opening	A real jar on a black marble pedestal, gold light up to the top of the screen. On scroll, the jar rises to its place on the stage	One canvas + a real cut-out photo
Film	The jar spins, the lid opens once, a hand lifts it, the lid closes in the hand. Four short AI videos turned into 121 still frames each	Kling 3.0 (Magnific) + ffmpeg
Scroll engine	Your scroll position picks which frame is on screen. Scroll down, the film plays. Scroll up, it rewinds	Plain JavaScript, no libraries
Cards	Four photos of different people, the jar always in the same spot. Between photos, a film where the jar jumps from one hand to the next	3 transition videos, 60 frames each
Text	Headlines fade and rise at exact points of the film	CSS + a 6-line timing function
Look	Warm black, gold, ivory. Archivo for the big words, Inter for the rest	Google Fonts, CSS variables

One HTML file of about 23 KB, no framework, no build step. The rest is images.

Step 1 - Pick the reference before any code

We took a product site where the product is on screen from start to end (a coaster that turns, floats, is held by a hand, then travels across photos of people). We watched it frame by frame, every half second, and wrote the scenes down. The rule we kept: **the product never leaves the screen until the end**.

Step 2 - Write the scenes like a short film

#	Scene	What happens
1	Opening	Small jar on a marble pedestal, luxury-ad lighting, giant brand name
2	The room fades	Dark warm background, jar alone in the center, gold edge on the lid
3	The jar spins	Front to back, so you see the ingredients
4	The lid opens, once	The pomade appears, smooth and untouched
5	The hand	A hand rises from below holding the open jar
6	The lid closes	In the hand
7	The jar travels	Four cards, four different people, the jar in the same spot every time
8	End	Jar front and center and one button

Do this on paper first. Every later step is just making one line of this table real.

Step 3 - Make the pictures (AI, with the real product inside)

Rules that saved us days:

1. **Start from a real photo of the product.** Cut it out and keep it as the truth. Never ask the AI to invent the product.
2. **Ask the image model to build the scene around the real jar** (we used Nano Banana Pro): marble pedestal, dark background, hand holding the jar. One approved image per scene before anything else.
3. **No faces.** Hands, beard from the chin down, the room. Faces look fake.
4. **Test videos at low resolution first (480p).** Only the approved test is rendered large.
5. **The lid opens one time.** Our first try failed because every clip started from the closed jar, so the lid opened five times while scrolling, and each clip invented a different pomade. The fix: make ONE still image of the open jar, approve it, and use it as the last frame of the lid clip and the first frame of the next ones.

Step 4 - Make the films (one clip per move)

Four clips, each starts on an approved image and ends on the next:

- spin (front to back, new back label with big ingredients)
- lid opens once
- hand rises and holds the open jar
- lid closes in the hand

Plus three **passages** between the photos. A passage prompt says: the jar jumps and stays in the center, always visible; the previous hand lowers; the scene transforms around the jar; the next hand rises and takes it. We tried three worse ways first (cut-out pasted over photos, photos swapping while the jar stood still, jumps that left one side and entered the other). Only a continuous video worked.

Step 5 - Turn each film into frames

```
ffmpeg -i spin.mp4 -vf "scale=1920:1080" -q:v 3 frames/spin/%03d.jpg
```

A 5-second clip at 24 fps gives 121 frames. We used 121 for the four main films and 60 for each passage. Total weight: about 33 MB of JPG. Name them 001.jpg, 002.jpg... so the code can count.

Step 6 - One tall page with a sticky screen

```
<section id="filme">      <!-- very tall: 2190vh -->
  <div id="tela">         <!-- sticky: stays on screen -->
    <canvas id="quadro" width="1920" height="1080"></canvas>
    ...text blocks...
  </div>
</section>
```

```
#filme{ height:2190vh; position:relative }
#tela { position:sticky; top:0; height:100vh; overflow:hidden }
```

The page is a long road. The screen is a window that stays still while you walk.

Step 7 - Scroll becomes progress, progress becomes a frame

Scroll is not linear in this site: slow where you need to see (the lid, the hand), fast elsewhere. We describe it as checkpoints: progress 0 to 1 against "viewport heights scrolled".

```
const TRECHOS = [[0,0],[.70,840],[.845,1190],[1,2090]];  // [progress, vh scrolled]
function progresso(){ const vh = scrollY/innerHeight*100;
  for (let i=1;i<TRECHOS.length;i++){
    const [p0,v0]=TRECHOS[i-1],[p1,v1]=TRECHOS[i];
    if (vh<=v1 || i===TRECHOS.length-1) return clamp(p0+(p1-p0)*clamp((vh-v0)/(v1-v0)));
  } return 1; }
```

Then a **smooth follower**, so frames glide instead of jumping:

```
let alvo = 0, atual = 0;
addEventListener('scroll', ()=>{ alvo = progresso(); }, {passive:true});
// every animation frame: atual += (alvo - atual) * .12;  then draw(atual)
```

Step 8 - Which film plays at which progress

```
if (p < .225)      img = emptyStage;                  // jar comes from the opening
else if (p < .44)  img = frame(spin,  range(p,.31,.41));      // spin forward
else if (p < .505) img = frame(spin,  1-range(p,.44,.50));    // spin back
else if (p < .71)  img = frame(lid,   range(p,.53,.62));      // lid opens
else if (p < .78)  img = frame(hand,  range(p,.71,.78));      // hand lifts
else               img = frame(close, range(p,.785,.835));    // lid closes
```

where `frame(list,t)` is `list[Math.round(clamp(t)*(N-1))]` and `range(p,a,b)` is `clamp((p-a)/(b-a))`. That is the whole engine. Draw it with `ctx.drawImage(img,0,0,1920,1080)`.

Step 9 - The opening is a separate canvas

The first screen is not a frame. A photo of the empty marble pedestal is drawn full screen, gold light is painted up to the top, and **the real jar is drawn on top**, with a blurred shadow under it. Between progress .12 and .22 the jar rises from the pedestal to the exact spot where the film starts, the room fades, and the stage canvas takes over. Because the jar is the same picture in both, there is no jump.

Step 10 - The camera

Before the lid opens, the camera pulls back so you can see the lid. Easy: draw the frame smaller (scale `k < 1`) and lower, with an oval fade on the edges so the border melts into the background.

```
const longe = smooth(range(p,.50,.53)) * (1 - smooth(range(p,.695,.71)));
drawFrame(img, 1 - .38*longe, 120*longe);
```

Step 11 - Text on a timeline

Each text block carries two numbers in the HTML: when it appears and when it leaves.

```
<div class="txt esq" data-de=".21" data-ate=".30"> ...Not just a pomade... </div>
```

```
for (const el of texts){
  const a = clamp(Math.min(range(p,de,de+.025), 1-range(p,ate-.025,ate)));
  el.style.opacity = a; el.style.translate = '0 ' + ((1-a)*24) + 'px';
}
```

Fade in with a small rise, fade out the same way. Nothing on top of the jar when the lid is opening.

Step 12 - The jar travels through the people

Four photos with the jar at exactly the same size and position (62% of the card width), measured on each picture. The middle card stays; what changes is the background. Between two photos, play the passage film in that card. Three side cards (previous, current, next) are drawn on small canvases of 600 x 800.

Step 13 - Phones

Portrait screens (width/height under 0.9) get a different fit: the jar fills about 85% of the width, the stage fades at top and bottom with a CSS mask, and the opening uses a taller photo. Same code, one `if`.

Step 14 - Preload everything, show a percentage

All frames load before the page starts: a small counter (0% to 100%) fades out when the last image is ready. Total images to count: frames x 4 + 2.

Step 15 - Publish

Put `index.html`, the `img` folder and the frame folders in one folder and deploy it to a static host (Cloudflare Pages: `npx wrangler pages deploy <folder> --project-name <name>`). No server, no database.

What we would do again

- Write the scenes first. Generate only what the scenes need.
- Approve one image before you animate it. Test videos small.
- Keep the product real. Let the AI build the world around it.
- Make the scroll slow where the story is.

Want a site like this? Barbelo Agency builds custom sites for brands, artists and businesses: www.BarbeloAgency.com